PlayerTracking Module

1. Overview

The **PlayerTracking** module is responsible for detecting, tracking, and selecting players from videos. It integrates YOLO for player detection and applies geometric constraints (court convex hull) to filter and validate detections. Additionally, it supports persistence through stub files for reproducibility and efficiency.

This module is designed as a core component of the project, providing consistent player bounding boxes and IDs across video frames.

2. Dependencies

- **Ultralytics YOLO**: for real-time object detection.
- **OpenCV**: for geometric operations, polygon tests, and drawing.
- **NumPy**: for numerical operations and coordinate transformations.
- **Pickle**: for serializing detection results (stubs).

3. Class: PlayerTracking

Initialization

PlayerTracking(model_path: str, max_dist: int = 70)

- **Parameters**:
    1. model_path: Path to the YOLO model weights.
    2. max_dist: Maximum distance (in pixels) to accept detections outside the court convex hull.

Private Methods

_hull(court_kp)

- **Purpose**: Compute the convex hull of court keypoints.
- **Parameters**: court_kp (list of keypoints).
- **Output**: Hull polygon as a NumPy array.

_ref_point_status(box, hull)

- **Purpose**: Evaluate whether a detection lies inside the court.
- **Parameters**:
    1. box: YOLO bounding box [x1, y1, x2, y2].
    2. hull: Convex hull of court keypoints.
- **Output**: (ref_point, status, distance)
    1. ref_point: Key point representing player position.

2. status: “inside” or “outside”.
3. distance: Minimum distance to hull if outside.

Player Selection Methods

choose_players(court_kp, player_dt, max_dist=None)

- Selects the two players most relevant to the game using distances to the court hull.
- Returns a list of selected track IDs.

pick_players(court_kp, player_dt)

- Filters all frames to retain only the two chosen players.
- Returns a list of dictionaries with selected players per frame.

Detection Methods

detect_player(frames, read_stub=False, stub_path=None)

- **Purpose**: Detect players across all frames.
- **Parameters**:
    1. frames: List of video frames.
    2. read_stub: Load results from pickle if available.
    3. stub_path: Path to save/load detections.
- **Output**: List of detections per frame.

detect_frame(frame)

- **Purpose**: Run YOLO detection for a single frame.
- **Returns**: Dictionary of {track_id: bounding_box} for detected players.

Visualization Methods

draw_boxes(video_frames, player_detections, court_kp, color_box=(0,255,0), show_court=False)

- **Purpose**: Draw player bounding boxes, reference points, and optional court hull on video frames.
- **Parameters**:
    1. video_frames: List of frames.
    2. player_detections: Detected players per frame.
    3. court_kp: Court keypoints.
    4. color_box: Color of bounding box (default green).
    5. show_court: Boolean, overlay hull if True.
- **Output**: Annotated video frames.

4. Notes

- The class can accept any compatible version of YOLO, such as YOLOv8, up to YOLOv11, providing flexibility to adapt to different versions of the model.
- pick_players ensures robustness by selecting players closest to the court, filtering out referees, crowd, or irrelevant detections.
- Works in tandem with **BallTracking**, **CourtDetection**, and **PlayerPose** modules in the main pipeline.